

WEEK-3 PRACTICAL

Task 3: Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

CODE:

```
root@Mani: ~
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Parent process started\n");
    printf("Parent PID : %d\n", getpid());
    printf("Parent PPID : %d\n", getppid());
    printf("State      : Running\n\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("Child process created\n");
        printf("Child PID   : %d\n", getpid());
        printf("Child PPID  : %d\n", getppid());
        printf("State      : Running\n");

        sleep(3);

        printf("Child PID %d\n", getpid());
        printf("State      : Terminated\n");

        exit(0);
    }
    else
    {
        // Parent process
        printf("\nParent process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);
        printf("State      : Waiting\n");

        wait(NULL);

        printf("Child process completed\n");
        printf("Parent State: Running\n");
        printf("Parent State: Terminated\n");
    }

    return 0;
}
```

OUTPUT:

```
root@Mani: ~  
root@Mani:~# nano ParentChild.c  
root@Mani:~# gcc ParentChild.c -o ParentChild  
root@Mani:~# ./ParentChild  
Parent process started  
Parent PID : 735  
Parent PPID : 387  
State : Running  
  
Parent process  
Parent PID : 735  
Child PID : 736  
State : Waiting  
Child process created  
Child PID : 736  
Child PPID : 735  
State : Running  
Child PID 736  
State : Terminated  
Child process completed  
Parent State: Running  
Parent State: Terminated  
root@Mani:~#
```

REPORT: The program successfully creates a child process using `fork()`. The PID and PPID of the parent and child processes are displayed, and the process states are observed during execution.

Task 4: Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as `ps`, `top`, and `/proc`. Document the observations CODE:

PS -EF:

```
root@Mani:~# ps -ef
UID      PID     PPID  C  STIME TTY          TIME CMD
root         1         0  0  04:58 ?        00:00:00 /sbin/init
root         2         1  0  04:58 hvc0     00:00:00 /init
root         6         2  0  04:58 hvc0     00:00:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
root        52         1  0  04:58 ?        00:00:00 /usr/lib/systemd/systemd-journald
root        99         1  0  04:58 ?        00:00:00 /usr/lib/systemd/systemd-udevd
systemd+   151       100  0  04:58 ?        00:00:00 /usr/lib/systemd/systemd-resolved
systemd+   168       100  0  04:58 ?        00:00:00 /usr/lib/systemd/systemd-timesyncd
root       174       100  0  04:58 ?        00:00:00 /usr/sbin/cron -f -P
message+   175       100  0  04:58 ?        00:00:00 dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
root      185       100  0  04:58 ?        00:00:00 /usr/lib/systemd/systemd-logind
syslog    195       100  0  04:58 ?        00:00:00 /usr/sbin/rsyslogd -n -iNONE
root     207       100  0  04:58 tty1     00:00:00 /sbin/agetty -o -p -- \u --noclear - linux
root     208       100  0  04:58 ?        00:00:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
root     253       100  0  04:58 ?        00:00:00 /usr/lib/systemd/systemd-timedated
root     294        20  0  04:58 ?        00:00:00 /init
root     295       294  0  04:58 ?        00:00:00 /init
root     296       295  0  04:58 pts/0    00:00:00 -bash
root     297       296  0  04:58 pts/1    00:00:00 /bin/login -f
root     340       100  0  04:58 ?        00:00:00 /usr/lib/systemd/systemd --user
root     341       340  0  04:58 ?        00:00:00 (sd-pam)
root     363       297  0  04:58 pts/1    00:00:00 -bash
root     435        99  0  04:58 ?        00:00:00 (udev-worker)
root     436        99  0  04:58 ?        00:00:00 (udev-worker)
root     449       296  0  04:58 pts/0    00:00:00 ps -ef
root@Mani:~#
```

```
root@Mani:~# ps -o pid,ppid,state,cmd
PID      PPID  S  CMD
296       295  S  -bash
593       296  R  ps -o pid,ppid,state,cmd
root@Mani:~#
```

PS:

TOP:

```
root@Mani:~# top
top - 04:59:03 up 2 min, 1 user, load average: 0.03, 0.01, 0.00
Tasks: 21 total, 1 running, 20 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.1 us, 0.1 sy, 0.0 ni, 99.9 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7658.1 total, 6016.0 free, 549.1 used, 1244.7 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 7109.1 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM    TIME+  COMMAND
    1 root        20   0  21648  13040  9716  S   0.3   0.2   0:00.46 systemd
    2 root        20   0  3180   2204  2072  S   0.0   0.0   0:00.00 init-systemd(Ub
    6 root        20   0   3196   2128  2008  S   0.0   0.0   0:00.00 init
   52 root       19  -1  42164  16016  14832  S   0.0   0.2   0:00.22 systemd-journal
   99 root       20   0  25008  6364   5120  S   0.0   0.1   0:00.08 systemd-udevd
  151 systemd+   20   0  21460  13220  10964  S   0.0   0.2   0:00.04 systemd-resolve
  168 systemd+   20   0  91028  7988   7020  S   0.0   0.1   0:00.03 systemd-timesyn
  174 root       20   0   4244   2684   2432  S   0.0   0.0   0:00.00 cron
  175 message+   20   0   9596  5320   4652  S   0.0   0.1   0:00.05 dbus-daemon
  185 root       20   0  17972  8828   7784  S   0.0   0.1   0:00.03 systemd-logind
  195 syslog    20   0  222516  5592   4560  S   0.0   0.1   0:00.04 rsyslogd
  207 root       20   0   3124   1976   1836  S   0.0   0.0   0:00.00 agetty
  208 root       20   0  107016  23020  13556  S   0.0   0.3   0:00.07 unattended-upgr
  294 root       20   0   3184   1108    980  S   0.0   0.0   0:00.00 SessionLeader
  295 root       20   0   3200   1244   1108  S   0.0   0.0   0:00.00 Relay(296)
  296 root       20   0   6080   5332   3652  S   0.0   0.1   0:00.01 bash
  297 root       20   0   6676   4496   3724  S   0.0   0.1   0:00.00 login
  340 root       20   0  20104  11448  9516  S   0.0   0.1   0:00.05 systemd
  341 root       20   0  21156   3496   1788  S   0.0   0.0   0:00.00 (sd-pam)
  363 root       20   0   6080   5276   3652  S   0.0   0.1   0:00.01 bash
  679 root       20   0   9340   5728   3508  R   0.0   0.1   0:00.00 top
```

REPORT: This is understanding process creation and process state transitions in Linux. The `fork()` system call was used to create a child process, while `ps`, `top`, and `/proc` were used to monitor process information and states. The PID and PPID helped identify the relationship between the parent and child processes.

